AI: Mastering the Shell with GitHub Copilot CLI

Student: Pacifique Kabanda
Course: SE103
Assignment: AI: Mastering the Shell with GitHub Copilot CLI
Platform: Ubuntu Linux

1. Objective

The objective of this activity was to use GitHub Copilot CLI to generate, understand, and evaluate a Git command for combining two local commits into one commit with a new message.

2. Prompt Engineering

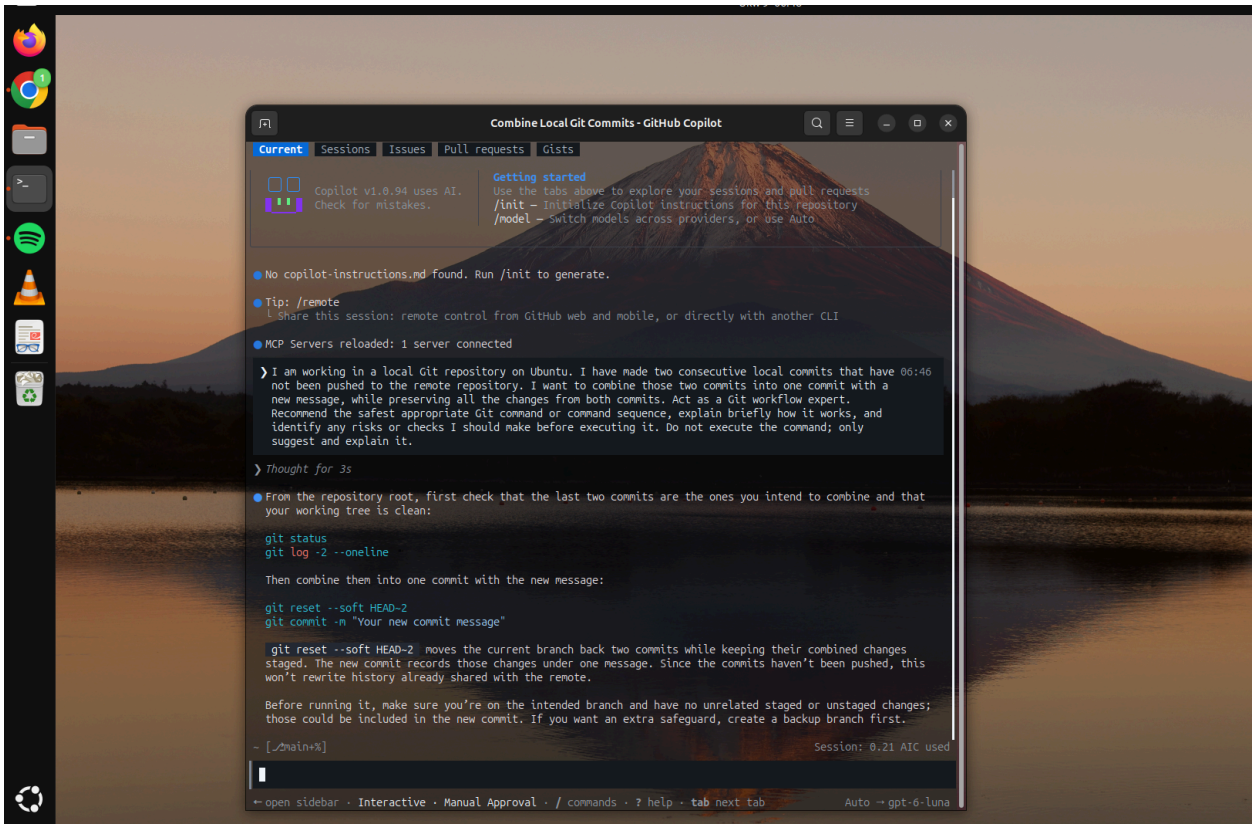
I provided GitHub Copilot CLI with a specific scenario: I had two consecutive local commits that had not been pushed to the remote repository, and I wanted to combine them into one commit while preserving all changes.

I asked Copilot to recommend an appropriate command sequence, explain how it works, and identify important checks and risks before execution. I also instructed it not to execute the command.

3. Evidence of Execution

Tool used: GitHub Copilot CLI

Screenshot of interaction:



Command suggested by Copilot:

From the repository root, first check that the last two commits are the ones you intend to combine and that your working tree is clean:

```
git status

git log -2 --oneline
```

Then combine them into one commit with the new message:

```
git reset --soft HEAD~2

git commit -m "Your new commit message"
```

`git reset --soft HEAD~2` moves the current branch back two commits while keeping their combined changes staged. The new commit records those changes under one message. Since the commits haven't been pushed, this won't rewrite history already shared with the remote.

Before running it, make sure you're on the intended branch and have no unrelated staged or unstaged changes; those could be included in the new commit. If you want an extra safeguard, create a backup branch first.

4. Accuracy Ranking and Reflection

Accuracy rating: 9/10

I would rate Copilot's response 9 out of 10 if its suggested command correctly combines the two local commits, preserves their changes, and creates a new commit with the requested message. Using an AI assistant can improve workflow efficiency by reducing the time spent searching for Git syntax and explaining unfamiliar commands. However, checking the repository status and understanding the effects of history-rewriting commands remain important before execution.

5. Conclusion

This activity demonstrated how GitHub Copilot CLI can translate a specific natural-language request into a practical Git workflow. I also learned the importance

of writing clear prompts, reviewing AI-generated commands, and checking their potential effects before running them.